

ODrive USB-CAN Adapter Datasheet

- [Electromechanical Specifications](#)
 - [CAD](#)
 - [Electrical](#)
 - [Mechanical](#)
 - [Environmental](#)
 - [Interface](#)
- [Pinout](#)
- [Connectors](#)
- [Wiring and Isolation](#)
- [Power](#)
- [Usage & Compatibility](#)
- [Unbinding from SocketCAN on Linux](#)



The ODrive USB-CAN adapter is a compact and robust CAN interface that connects to a host via USB-C. It's designed for use with ODrive products, but is compatible with any CAN 2.0B device.

The USB-CAN adapter features dual CAN connectors, with JST-GH for plug-and-play wiring to ODrive devices, and a screw terminal for flexibility in all wiring situations and other CAN applications.

The USB-CAN adapter is compatible with CAN 2.0B, up to 1Mbit/s. It shows up as a native SocketCAN device under Linux, enabling support for can-utils as well as a wide variety of programming languages. Additionally, it can be used with python-can under Linux, Windows, and Mac OS.

An onboard termination resistor switch allows for easy termination of a CAN bus, and onboard indicator LEDs report send/recieve data activity for user feedback.

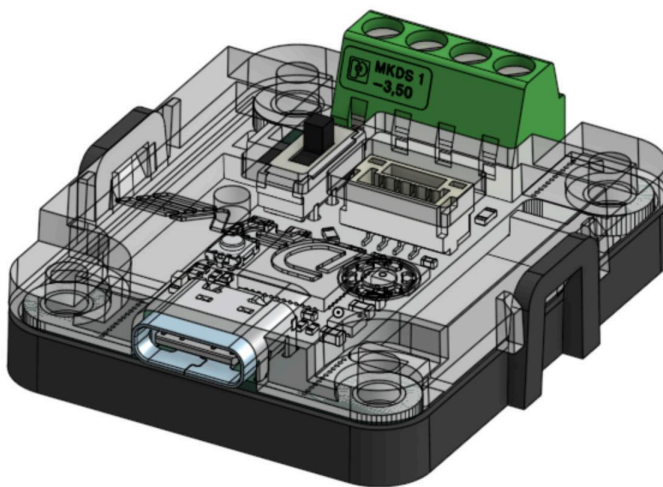
5V power from USB is supplied on the VBUS line, and an anti-backfeed circuit allows up to 30V external.

Preassembled wire harnesses to connect the USB-CAN adapter to an ODrive S1 or Micro are available on the [shop page](#).

Electromechanical Specifications

CAD

The ODrive USB-CAN Adapter CAD model is available on the [ODrive USB-CAN Adapter OnShape](#) page.



Electrical

Note

All specifications are in $0^{\circ}\text{C} \leq T_A \leq 40^{\circ}\text{C}$ unless otherwise noted.

Specification	Value	Notes
Power Input	4.5 – 5.5V	Over USB Type-C
Signaling Protocol	CAN 2.0B	10 Kbit/s to 1 Mbit/s
ESD Protection	$\pm 15\text{ kV}$ $\pm 15\text{ kV}$ $\pm 8\text{ kV}$ $\pm 30\text{ kV}$	USB Power Line, IEC 61000-4-2 USB Data Lines, IEC 61000-4-2 CAN Data Lines, IEC 61000-4-2 VBUS Power Line, IEC 61000-4-2
CANH/CANL Max Voltage	$\pm 58\text{ V}$	Abs max, referenced to device GND
Current Output	500 mA	USB Supply over VBUS line, fused

Mechanical

Specification	Value	Notes
Weight	11.8g 5.8g	With case Without case
Width Length Height	36.14 mm 40.16 mm 13.65 mm	With case
Bolt Pattern	25 mm x 32 mm	M2.5 Clearance

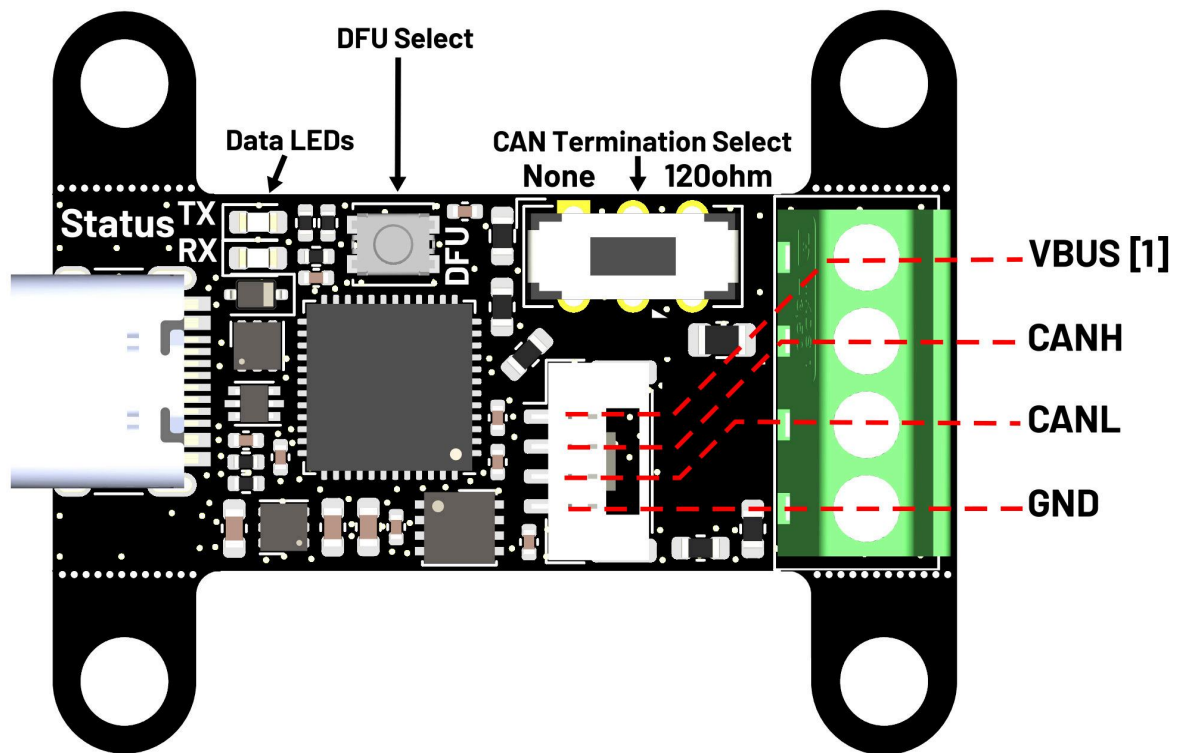
Environmental

Specification	Value	Notes
Humidity	Non-Condensing	
Ingress Protection	IP20	With case

Interface

Specification	Value	Notes
USB Speed	12Mbps	USB 2.0 FS
CAN Baudrate	10Kbit/s – 1Mbit/s	
TX Buffer Size	64	CAN Frames
RX Buffer Size	3	CAN Frames
Device Protocol	gs_usb compatible	

Pinout



- [1] VBUS is 4.75V nominal output, and tolerant of up to 30V. See [Power](#) section for details.

Connectors

Connector	Type	Mating housing	Mating Crimp
CAN JST-GH	JST-GH 4-pos	GHR-04V-S	SSHL-002T-P0.2
CAN Screw Terminal	Screw Terminal 4-pos	N/A	16-28 AWG Wire

Wiring and Isolation

Note that while the fusing on the USB-CAN adapter provides nominal protection against ground loops, care still must be taken. As well, the USB-CAN adapter must share a ground domain with the connected CAN devices.

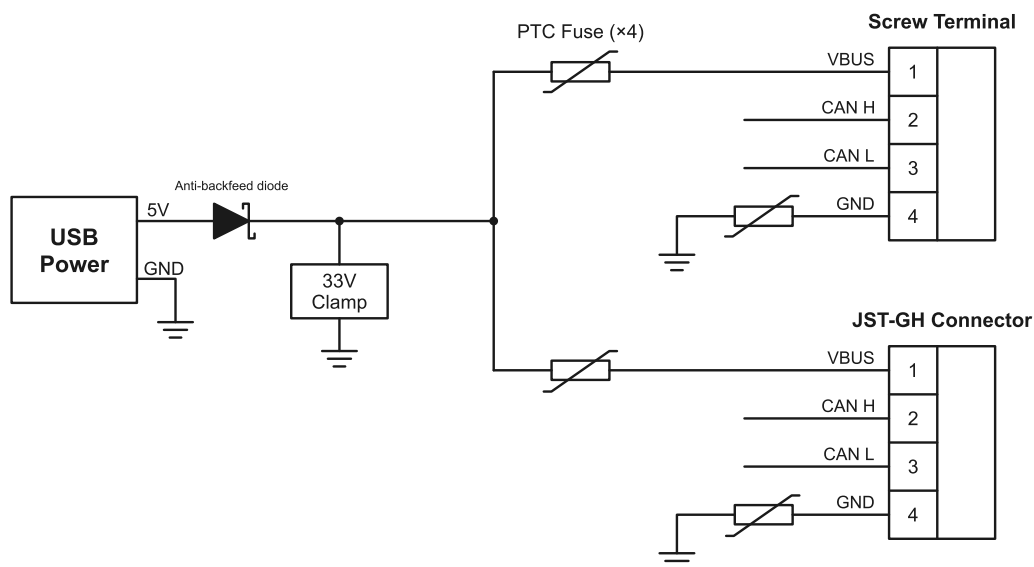
This means that in many cases, it is required to use a USB isolator with the USB-CAN adapter, and/or to have an external ground tie between the USB-CAN adapter and the system DC-.

For general information and wiring, please see the [CAN grounding diagram](#).

Power

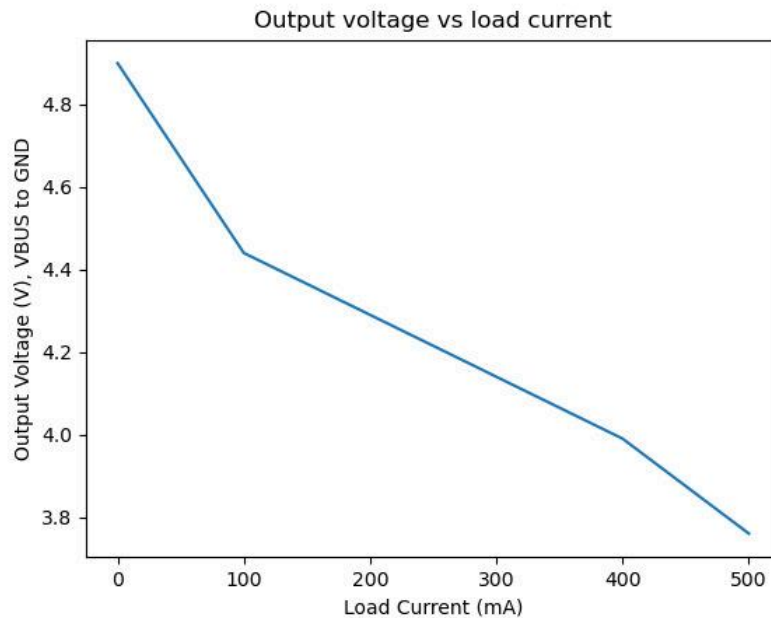
The USB-CAN adapter provides a useful USB-derived voltage output on the VBUS pin / CAN power line. This output is diode-protected, and will not backfeed to the USB power, so higher voltages (up to 30V) are allowed on the VBUS line. This ensures compatibility for CAN devices deriving power from the power/ground wires in the CAN cable (up to 30V), such as an ODrive Pro or S1 using the 12V standby power input over the CAN cable, or an ODrive Micro with [CAN power lines bridged to DC+/-](#).

The USB-CAN adapter is additionally designed to protect the upstream USB-connected computer in the event of catastrophic failure, such as an overvoltage spike or sustained overcurrent on the output power connection.



The USB power is fed through a diode, and each VBUS / GND output is individually fused with a 500mA PTC (resettable) fuse. A circuit clamps the VBUS voltage to 33V, preventing damage to the upstream USB-connected computer.

In testing, a typical no-load VBUS output voltage is 4.8-4.9V. However, the PTC fuse elements add resistance to both the VBUS output and GND return, causing voltage drop with increasing current draw from the VBUS/GND outputs. A chart of nominal output voltage versus load current is provided below:



Usage & Compatibility

The ODrive USB-CAN Adapter is the interface of choice for future ODrive software, such as `odrivetool` and GUI over CAN bus. These features are under active development, and are expected for public beta Q4 2024. In the meantime, please [contact us](#) if you are interested in beta testing.

The USB-CAN Adapter uses a `gs_usb` compatible firmware. This means that under Linux, it shows up as a native SocketCAN device, and thus a generic networking port (similar to Ethernet or WiFi). Unfortunately, Windows and Mac OS do not support native CAN devices, however the USB-CAN adapter can still be used with any CAN library with `gs_usb` support. We typically recommend use with the `python-can` library, however others may exist.

Linux

Windows / Mac OS

Upon connecting the ODrive USB-CAN adapter, it will show up as a CAN port such as `can0`. This can be seen with the `ip addr` command. Example output:

```

1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: wlp0s20f3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc
noqueue state UP group default qlen 1000
    link/ether 8c:f8:c5:3b:6c:35 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.213/24 brd 192.168.1.255 scope global dynamic
noprefixroute wlp0s20f3
        valid_lft 73339sec preferred_lft 73339sec
    inet6 fe80::c915:97ab:92b3:42cb/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
15: can0: <NOARP,ECHO> mtu 16 qdisc noop state DOWN group default
qlen 10
    link/can

```

To enable the USB-CAN adapter at a specified baudrate (here, 250000 baud), run the following commands:

```

sudo ip link set can0 type can bitrate 250000
sudo ip link set up can0

```

If successful, the `can0` interface output of `ip addr` will look similarly to the below:

```

15: can0: <NOARP,UP,LOWER_UP,ECHO> mtu 16 qdisc pfifo_fast state UP
group default qlen 10
    link/can

```

From here, it can be used with any SocketCAN compatible library or tool, such as [can-utils](#), or in nearly any programming language with a SocketCAN library, such as [python](#), [C/C++](#), [Rust](#), [Javascript](#), etc.

For example, in Python, using python-can:

```

import can
bus = can.Bus(interface='socketcan', channel='can0')

```

Unbinding from SocketCAN on Linux

By default, Linux will use the USB-CAN adapter as a native SocketCAN device (e.g. `can0`). However, this prevents the Web GUI from being able to access and manage the USB-CAN adapter. In order to use the Web GUI over CAN on Linux, we need to unbind the SocketCAN kernel driver and change the device permissions:

Command Line

```
# Unbind the gs_usb kernel driver from all connected USB-CAN Adapters
for d in /sys/bus/usb/drivers/gs_usb/*:*; do
    [ -e "$d" ] && echo "$(basename "$d")" | sudo tee
    /sys/bus/usb/drivers/gs_usb/unbind
done

# Change device permissions to allow access by the browser
for dev in /dev/bus/usb/*/*; do
    if lsusb -s ${dev##*/} 2>/dev/null | grep -q "1d50:606f"; then
        sudo chmod 666 "$dev"
    fi
done
```

This will make the USB-CAN Adapter unavailable for other applications that use it via SocketCAN.

To undo this and return ownership to SocketCAN, simply replug the USB-CAN Adapter or reboot the host.

Note

While SocketCAN is the recommended interface on Linux outside the Web GUI, both `odrivetool` and the Python package can access the USB-CAN adapter directly after unbinding.